

Tutorial Overview: Setting Up a CRUD Project with Express.js, MongoDB, and EJS

1. Understanding CRUD Operations

When working with Express.js and a database, there are **four core operations**:

1. **Create** – Add new data to the database
2. **Read** – Fetch data from the database
3. **Update** – Modify existing records
4. **Delete** – Remove records

Together, these operations form a **CRUD system**.

2. Project Roadmap

The CRUD project is divided into multiple steps:

1. Step 1

- Set up the HTML template in Express.js
- Configure EJS, routes, middleware, and static files

2. Step 2

- Connect MongoDB
- Read data from MongoDB
- Display data in EJS templates

3. Step 3

- Add, update, and delete data from MongoDB

4. Step 4

- Refactor code using **MVC architecture**
 - Add proper **error handling**
-

3. CRUD Application Pages (UI Overview)

The template includes the following pages:

- **Home Page** Displays a list of all contacts
- **Add New Contact Page** Form to create a new contact
- **View Contact Page** Displays detailed information of a single contact
- **Update Contact Page** Form to edit an existing contact
- **Delete Button** Deletes a contact (logic added later)

4. Creating the Project Folder

1. Open **VS Code**
2. Create a new folder for the project Example:

```
contact-app
```

3. Open the folder in VS Code
-

5. Initializing Node.js Project

Open the terminal and run:

```
npm init -y
```

This creates the `package.json` file.

6. Installing Required Packages

Install Express, Nodemon, and EJS:

```
npm install express nodemon ejs
```

7. Configuring Nodemon

Edit `package.json` and add a start script:

```
"scripts": {  
  "start": "nodemon index.js"  
}
```

8. Creating the Main Server File

Create a file named:

```
index.js
```

Basic Express server setup:

```
const express = require("express");
const app = express();

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Run the server:

```
npm run start
```

9. Defining Application Routes

Seven routes are created for the CRUD system:

Purpose	Method	Route
Home (All Contacts)	GET	/
View Single Contact	GET	/show-contact
Open Add Contact Form	GET	/add-contact
Save New Contact	POST	/add-contact
Open Update Contact Form	GET	/update-contact
Save Updated Contact	POST	/update-contact
Delete Contact	GET/POST	/delete-contact

10. Configuring Middleware

1. Set EJS as View Engine

```
app.set("view engine", "ejs");
```

2. Parse Form Data

```
app.use(express.urlencoded({ extended: true }));
```

3. Serve Static Files

```
app.use(express.static("public"));
```

11. Project Folder Structure

```
contact-app/
├── index.js
├── package.json
├── views/
│   ├── home.ejs
│   ├── add-contact.ejs
│   ├── update-contact.ejs
│   ├── show-contact.ejs
│   └── partials/
│       ├── header.ejs
│       └── footer.ejs
└── public/
    ├── bootstrap.css
    └── custom.css
```

12. Creating EJS Templates

Four EJS files are created:

1. **home.ejs** – Displays all contacts
2. **add-contact.ejs** – Add contact form
3. **update-contact.ejs** – Update contact form
4. **show-contact.ejs** – Single contact details

HTML templates downloaded from the website are copied into these files.

13. Using EJS Partials (Header & Footer)

To avoid repeating HTML code:

Header Partial (`partials/header.ejs`)

- `<!DOCTYPE html>`
- `<head>` section
- Navigation bar
- CSS links

Footer Partial (`partials/footer.ejs`)

- Closing `</body>` and `</html>` tags

Including Partials in EJS Files

```
<%- include("partials/header") %>

<main>
  <!-- Page Content -->
</main>

<%- include("partials/footer") %>
```

14. Linking Static CSS Files

CSS files are placed inside the `public` folder.

In `header.ejs`:

```
<link rel="stylesheet" href="/bootstrap.css">
<link rel="stylesheet" href="/custom.css">
```

Express automatically serves these files from `public`.

15. Rendering Pages from Routes

Example route rendering:

```
app.get("/", (req, res) => {
  res.render("home");
});

app.get("/add-contact", (req, res) => {
  res.render("add-contact");
});

app.get("/update-contact", (req, res) => {
  res.render("update-contact");
});

app.get("/show-contact", (req, res) => {
  res.render("show-contact");
});
```

16. Updating Navigation Links

All HTML links are updated to use **Express routes**, not `.html` files.

Examples:

- Add Contact → `/add-contact`
 - View Contact → `/show-contact`
 - Update Contact → `/update-contact`
 - Cancel Button → `/`
-

17. Final Output Verification

After refreshing the browser:

- Home page loads correctly
 - CSS is applied
 - Add, View, and Update pages open correctly
 - Cancel buttons navigate back to Home
 - Delete button exists (logic handled later)
-

18. Key Takeaways

- Express.js server setup completed
 - EJS templating integrated
 - Routes structured for CRUD
 - Static assets configured
 - Reusable layout created using partials
-